



University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

CRT-Royale [MSL]

Portierung von CRT-Royale in die Metal Shading Language
sowie Implementierung in RetroVisor

Projektdokumentation

Software-Projekt
Sommersemester 2026

Betreuer: Prof. Dr. Hoffmann

Bearbeiter: Devin Uyan

Repository: <https://github.com/snowIsNotAvaiable/crt-royale-msl>

Inhaltsverzeichnis

1	Einleitung	3
1.1	Motivation	3
1.2	Projektziel	3
1.3	Abgrenzung	3
2	Grundlagen	4
2.1	GPU-Programmierung und Compute Shader	4
2.2	Metal Shading Language (MSL)	4
2.3	GLSL vs. MSL – Wesentliche Unterschiede	5
3	Ist-Analyse: CRT-Royale	5
3.1	Überblick	5
3.2	Die 12-Pass-Pipeline	6
3.3	Quellstruktur	6
3.4	Konfigurierbare Parameter (Auszug)	7
4	Zielarchitektur: RetroVisor	7
4.1	Überblick	7
4.2	Shader-Integrationsmuster	8
4.3	Render-Pipeline	8
5	Technologieauswahl	9
6	Portierungsstrategie	9
6.1	Vorgehen	9
6.2	Übersetzungsregeln	9
6.3	Herausforderungen	11
7	Implementierung	11
7.1	Aktueller Stand	11
7.2	Pass 1: Code-Beispiel (MSL)	12
7.3	Swift-Integration	12
7.4	Pass 2 – Vertikale Scanlines	13
7.5	Pass 3 – Phosphor-Maske	14
7.6	BLOOM_APPROX (Slang Pass 2)	16
7.7	BRIGHTPASS (Slang Pass 8)	16
7.8	Multi-Mask-Type-Support	17
7.9	Bloom-Kette – BLOOM_V + BLOOM_FINAL (Slang Pass 9+10)	18
7.10	Pass 4 – Geometry, Border, Final Encode (voller Port)	20
7.11	Debug-Pipeline	22
7.12	Snapshot-Export	22
7.13	Headless Validierungs-Pipeline	22
7.14	Vergleichswerkzeug <code>compare.py</code>	24
7.15	Reference-Pipeline via <code>librashader-cli</code>	25
7.16	Dateien im Projekt	28
8	Validierungskonzept	28

8.1	Methodik	28
8.2	Testmuster	29
9	Evaluierung (Phase 4)	30
9.1	Performance-Messung	30
9.2	Speicherverbrauch (VRAM)	30
9.3	Qualitätsvergleich vs. Slang-Original	31
9.4	Qualitativer Vergleich vs. Sankara / CRT-Easy	31
9.5	Bekannte Limitationen	32
10	Projektplanung	32
10.1	Phasenübersicht	32
10.2	Detailplan Phase 1: Portierung	32
11	Muss- und Kann-Kriterien	33
12	Glossar	34

1 Einleitung

1.1 Motivation

Retro-Gaming erfreut sich wachsender Beliebtheit. Emulatoren wie RetroArch oder OpenEmu ermöglichen es, klassische Konsolenspiele auf moderner Hardware zu spielen. Dabei geht jedoch ein wesentliches visuelles Merkmal verloren: das charakteristische Bild eines **CRT-Monitors** (Cathode Ray Tube) mit seinen Scanlines, dem Phosphor-Glow und der typischen Farbwiedergabe.

CRT-Royale von TroggleMonkey ist einer der aufwendigsten und physikalisch genauesten CRT-Shader. Er simuliert in einer **12-Pass-Pipeline** nahezu alle optischen Eigenschaften eines CRT-Bildschirms – von der Gamma-Korrektur über Phosphormasken bis hin zu Bloom und Geometrieverzerrung. Allerdings existiert CRT-Royale bisher nur für Slang/GLSL (libretro), Cg und HLSL (ReShade). Eine native Portierung auf **Apple Metal (MSL)** fehlt vollständig.

1.2 Projektziel

Ziel dieses Semesterprojekts ist die vollständige Portierung des CRT-Royale-Shaders von Slang/GLSL in die **Metal Shading Language (MSL)** und dessen Integration in **RetroVisor**, eine macOS-Anwendung von Prof. Dr. Hoffmann, die GPU-basierte Bildeffekte als Echtzeit-Overlay auf beliebige Fenster anwendet.

Das Projekt gliedert sich in vier Phasen:

1. **Portierung:** Übersetzung aller 13 Shader-Passes von GLSL nach MSL
2. **Integration:** Einbindung der MSL-Shader in die RetroVisor-Architektur
3. **Validierung:** Visueller Vergleich mit dem GLSL-Original (Comparison-Images)
4. **Evaluierung:** Qualitäts- und Performancebewertung gegenüber dem Original

1.3 Abgrenzung

- Es wird **kein eigener Emulator** entwickelt – RetroVisor ist eine bestehende Anwendung.
- Die Portierung beschränkt sich auf den **CRT-Royale-Shader**.
- Zielplattform ist ausschließlich **macOS mit Apple Silicon** (Metal API).
- Machine-Learning-basierte Upscaling-Verfahren (z. B. FSR, DLSS) sind nicht Gegenstand.

2 Grundlagen

2.1 GPU-Programmierung und Compute Shader

Moderne GPUs sind massiv parallele Prozessoren, die Tausende von Threads gleichzeitig ausführen können. Im Gegensatz zu CPUs, die für sequentielle Aufgaben optimiert sind, eignen sich GPUs ideal für Aufgaben, bei denen **derselbe Algorithmus auf viele Datenpunkte gleichzeitig** angewendet wird (SIMD – Single Instruction, Multiple Data).

Compute Shader sind Programme, die direkt auf der GPU laufen und nicht an die klassische Grafikpipeline (Vertex → Fragment) gebunden sind. Sie arbeiten auf einem flexiblen Grid aus *Threadgroups*:

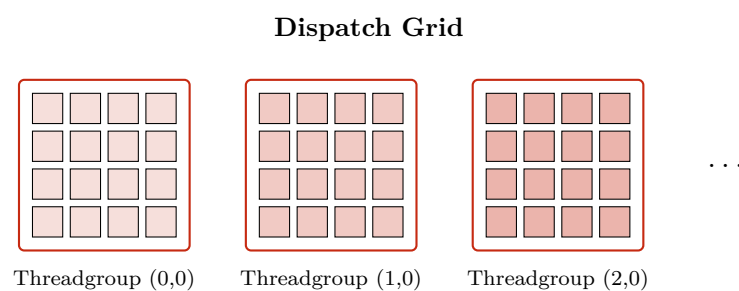


Abbildung 1: Compute-Shader-Dispatch:

Das Bild wird in Threadgroups unterteilt, jeder Thread bearbeitet ein Pixel

Jeder Thread kennt seine Position im Grid (`thread_position_in_grid`) und bearbeitet typischerweise genau ein Pixel. RetroVisor verwendet **16×16 Threadgroups** – jede Gruppe bearbeitet also einen 16×16-Pixel-Block des Ausgabebildes.

2.2 Metal Shading Language (MSL)

MSL ist Apples GPU-Programmiersprache, basierend auf C++14 mit GPU-spezifischen Erweiterungen.

Zentrale Konzepte:

Konzept	Beschreibung
<code>kernel</code>	Einstiegspunkt für Compute Shader (äquivalent zu <code>main()</code> in GLSL)
<code>texture2d<T></code>	2D-Textur mit Typparameter (z. B. <code>float</code> oder <code>half</code>)
<code>sampler</code>	Konfiguriert Textur-Sampling (Filter, Adressierung)
<code>float2/3/4</code>	Vektortypen (äquivalent zu GLSL <code>vec2/3/4</code>)
<code>constant T&</code>	Uniform-Daten (read-only, von der CPU übergeben)
<code>[[buffer(n)]]</code>	Attribut: bindet Parameter an Buffer-Slot <i>n</i>
<code>[[texture(n)]]</code>	Attribut: bindet Textur an Slot <i>n</i>
<code>thread_position_in_grid</code>	Globale Thread-ID (welches Pixel bearbeitet wird)

Tabelle 1: Zentrale MSL-Konzepte

2.3 GLSL vs. MSL – Wesentliche Unterschiede

Die Portierung von GLSL nach MSL erfordert systematische Übersetzungen:

Aspekt	GLSL / Slang	Metal (MSL)
Vektortypen	<code>vec2, vec3, vec4</code>	<code>float2, float3, float4</code>
Textur-Zugriff	<code>texture(sampler2D, uv)</code>	<code>tex.sample(sam, uv)</code>
Textur + Sampler	Kombiniert (<code>sampler2D</code>)	Getrennt (<code>texture2d + sampler</code>)
Uniforms	<code>uniform float x;</code>	<code>constant Uniforms &u [[buffer(0)]]</code>
Fragment-Koord.	<code>gl_FragCoord</code>	<code>thread_position_in_grid</code>
Shader-Typ	Fragment Shader	Compute Kernel
Modulo	<code>mod(x, y)</code>	<code>x - y * trunc(x/y)</code> (manuell)
Ausgabe	<code>FragColor = ...</code>	<code>outTexture.write(color, gid)</code>

Tabelle 2: GLSL-zu-MSL-Übersetzungstabelle

Ein besonders wichtiger Unterschied: GLSL-Shader nutzen die **Fragment-Shader-Pipeline** (ein Dreieck wird gerastert, pro Pixel läuft der Shader). RetroVisor hingegen setzt auf **Compute Shader**, bei denen das Dispatch-Grid explizit berechnet wird. Dies bietet mehr Flexibilität, erfordert aber manuelle Bounds-Checks.

3 Ist-Analyse: CRT-Royale

3.1 Überblick

CRT-Royale ist ein Open-Source-Shader (GPL v2+) von *TroggleMonkey*, der in der **libretro/slang-shaders**-Sammlung gepflegt wird. Er gilt als einer der physikalisch genauesten CRT-Simulatoren und modelliert folgende optische Phänomene:

- **Gamma-Linearisierung:** Umrechnung vom CRT-Gamma-Farbraum in lineares Licht
- **Scanlines:** Horizontale Helligkeitsmodulation (Zeilen eines Kathodenstrahlröhren-Bildes)
- **Phosphormasken:** RGB-Subpixel-Muster (Aperture Gitter, Slot Mask, Shadow Mask)
- **Bloom / Halation:** Lichtüberstrahlung durch die Glasscheibe des Monitors
- **Konvergenz:** Leichte Verschiebung der RGB-Elektronenstrahlen
- **Interlacing:** Halbbilder-Simulation (240p/480i)
- **Geometrieverzerrung:** Gewölbte Bildfläche eines CRT-Monitors

3.2 Die 12-Pass-Pipeline

CRT-Royale verarbeitet das Eingabebild in 13 aufeinanderfolgenden Passes. Jeder Pass liest die Ausgabe des vorherigen und schreibt in eine Zwischentextur:

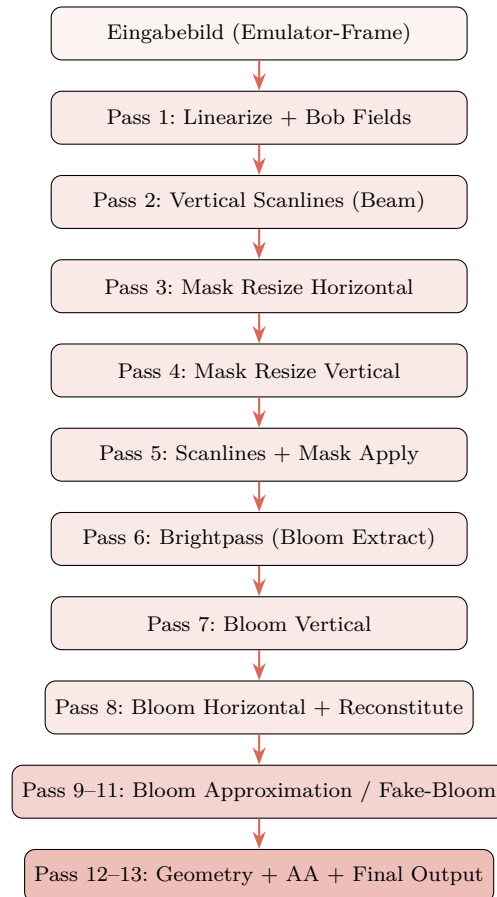


Abbildung 2: CRT-Royale 12-Pass-Pipeline – Datenfluss vom Eingabebild zum fertigen CRT-Effekt

3.3 Quellstruktur

Die relevanten Quelldateien befinden sich unter `crt/shaders/crt-royale/src/`:

Datei	Funktion
<code>bind-shader-params.h</code>	Alle ~45 konfigurierbaren Parameter
<code>gamma-management.h</code>	Gamma-Linearisierung und -Encoding
<code>scanline-functions.h</code>	Beam-Profil, Scanline-Breite, Interlace-Erkennung
<code>phosphor-mask-resizing.h</code>	Lanczos-Sinc-Resampling der Phosphormaske
<code>bloom-functions.h</code>	Bloom-Berechnung und -Extraktion
<code>geometry-functions.h</code>	Bildschirmkrümmung, Pin-Cushion-Verzerrung
<code>tex2Dantialias.h</code>	Anti-Aliasing-Algorithmen (~80 KB)
<code>derived-settings-and-constants.h</code>	Berechnete Konstanten aus User-Settings
<code>user-settings.h</code>	Nutzer-Konfiguration und GPU-Capability-Flags

Tabelle 3: Wichtige Header-Dateien des CRT-Royale-Originals

3.4 Konfigurierbare Parameter (Auszug)

CRT-Royale bietet ca. 45 Parameter, die das Erscheinungsbild steuern. Die wichtigsten:

Parameter	Default	Beschreibung
crt_gamma	2.5	Gamma des simulierten CRT-Monitors
lcd_gamma	2.2	Gamma des realen Ausgabedisplays
mask_type	1	Phosphormasken-Typ (0=Grille, 1=Slot, 2=Shadow)
interlace_detect	1	Automatische Interlace-Erkennung
beam_min_sigma	0.02	Minimale Scanline-Breite (Beam)
beam_max_sigma	0.30	Maximale Scanline-Breite (Beam)
bloom_underestimate	0.80	Bloom-Intensität
geom_mode	0	Geometrie-Modus (0=flat, 1-3=curved)
geom_radius	2.0	Krümmungsradius des simulierten CRT
convergence_offset	0.0	RGB-Konvergenz-Verschiebung

Tabelle 4: Wichtige CRT-Royale-Parameter

4 Zielarchitektur: RetroVisor

4.1 Überblick

RetroVisor ist eine macOS-Anwendung von Prof. Dr. Hoffmann, die GPU-basierte Bildeffekte als Echtzeit-Overlay auf beliebige Bildschirmfenster anwendet. Die Anwendung nutzt Apples **ScreenCaptureKit** zur Fensteraufnahme und die **Metal API** für die GPU-beschleunigte Bildverarbeitung.

RetroVisor enthält bereits zwei integrierte Shader:

- **Sankara**: Multi-Pass-Effekt mit Split/Composite/CRT/DotMask-Kernels
- **CRT-Easy**: Einfacher Single-Pass-CRT-Effekt

CRT-Royale MSL wird als **dritter Shader** in RetroVisor integriert.

4.2 Shader-Integrationsmuster

Jeder Shader in RetroVisor besteht aus zwei Dateien:

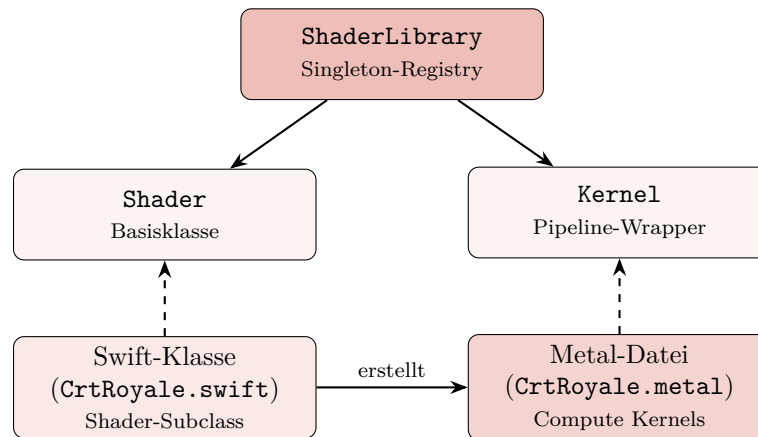


Abbildung 3: RetroVisor Shader-Integrationsmuster

1. Swift-Klasse (erbt von Shader):

- Definiert **Uniforms**-Struct (gespiegelt zum Metal-Pendant)
- Erstellt **Kernel**-Objekte für jeden Pass
- Implementiert **apply()** zur Orchestrierung der Multi-Pass-Pipeline
- Definiert UI-Settings (**ShaderSetting**) für Nutzerinteraktion

2. Metal-Datei (GPU-Code):

- Enthält Compute-Kernel-Funktionen (**kernel void**)
- Liest Eingabetexturen, schreibt Ausgabebetexturen
- Empfängt Uniforms über **constant**-Buffer

4.3 Render-Pipeline

Für jeden Frame durchläuft RetroVisor folgenden Ablauf:

1. **ScreenCaptureKit** liefert ein neues Frame des erfassten Fensters
2. Die **apply()**-Methode des aktiven Shaders wird aufgerufen
3. Für jeden Pass: ein **Kernel.apply()**-Aufruf dispatcht den Compute Shader
4. Jeder Kernel liest von einer Quell-Textur und schreibt in eine Ziel-Textur
5. Das finale Ergebnis wird als Overlay über das Originalfenster gelegt

Bei CRT-Royale bedeutet dies: **13 Kernel-Dispatches pro Frame**, mit 12 Zwischentexturen.

5 Technologieauswahl

Komponente	Technologie	Begründung
GPU-Sprache	Metal Shading Language	Einzige native GPU-API auf macOS; direkte Integration in RetroVisor
Host-Sprache	Swift	RetroVisor ist in Swift geschrieben; Metal-Framework nativ verfügbar
Build-System	Xcode	Standard für macOS-Entwicklung; Metal-Compiler integriert
Quellformat	Slang / GLSL	CRT-Royale-Originalformat in der libretro-Shader-Sammlung
Versionskontrolle	Git / GitHub	Kollaboration und Nachvollziehbarkeit
Testquelle	OpenEmu + 240p Test Suite	NES-Emulator mit standardisierten Testmustern für Shader-Validierung
Referenz	librasher- <code>cli</code> (Slang)	Slang-Original via Metal-Backend (Per-Pass-Snapshots)

Tabelle 5: Technologieübersicht

6 Portierungsstrategie

6.1 Vorgehen

Die Portierung erfolgt **inkrementell, pass-weise**:

1. Jeden Pass einzeln von Slang/GLSL nach MSL übersetzen
2. In RetroVisor integrieren und mit Zwischentexturen verknüpfen
3. Visuell gegen das GLSL-Original validieren (Comparison-Images)
4. Erst nach erfolgreicher Validierung zum nächsten Pass übergehen

Diese Strategie ermöglicht frühes Feedback: bereits nach Pass 1 (Linearisierung) ist ein sichtbarer Effekt vorhanden und testbar.

6.2 Übersetzungsregeln

Folgende systematische Transformationen werden bei jeder Pass-Portierung angewendet:

1. **Vektortypen umbenennen:** `vec2` → `float2`, `vec3` → `float3`, etc.
2. **Textur-Zugriff trennen:** `texture(tex, uv)` → `tex.sample(sam, uv)`
3. **Uniforms bündeln:** Einzelne `uniform`-Deklarationen → `constant Uniforms&`
4. **Fragment → Compute:** `gl_FragCoord` → `thread_position_in_grid`
5. **Ausgabe umschreiben:** `FragColor = ...` → `outTex.write(color, gid)`
6. **Bounds-Check einfügen:** Compute Shader müssen prüfen, ob `gid` innerhalb der Textur liegt

7. **GLSL-Funktionen ersetzen:** `mod()` manuell, `fract()`/`clamp()`/`mix()` sind identisch

6.3 Herausforderungen

- **Multi-Pass-Orchestrierung:** Im Original steuert eine `.glslp`-Preset-Datei die Verkettung. In RetroVisor muss die `apply()`-Methode dies manuell übernehmen.
- **Texture-Size-Semantik:** GLSL-Presets definieren `scale_type` (source/viewport/absolute) pro Pass. Diese Müsse in Swift korrekt berechnet werden.
- **Phosphormasken-Texturen:** PNG-Tile-Texturen müssen als `MTLTexture` geladen und korrekt gesampelt werden.
- **tex2Dantialias.h:** Die größte Header-Datei (~80 KB) mit komplexen Anti-Aliasing-Algorithmen.
- **Präzisionsunterschiede:** GLSL verwendet standardmäßig `mediump/highp`; MSL bietet `half` und `float`. Numerische Abweichungen müssen erkannt und behandelt werden.

7 Implementierung

7.1 Aktueller Stand

Zum aktuellen Zeitpunkt sind **Pass 1 (Linearize + Bob Fields)** und **Pass 2 (Vertical Scanlines)** vollständig portiert und in RetroVisor integriert. Zusätzlich existiert eine Debug-Pipeline (per-Pass Picker) sowie ein PNG-Snapshot-Export, der die spätere Pixel-Diff-Validierung gegen RetroArch vorbereitet.

Pass 1 umfasst:

- **Gamma-Linearisierung:**
`pow(color.rgb, crt_gamma)` – Umrechnung vom nicht-linearen CRT-Farbraum in lineares Licht
- **Interlace-Erkennung:**
Automatische Erkennung von 480i-Signalen anhand der Texturhöhe (> 288 Zeilen)
- **Bob-Field-Deinterlacing:**
Bei Interlace-Content wird abwechselnd das gerade/ungerade Halbbild angezeigt
- **Final-Encode:**
Rück-Übersetzung von linearem Licht in den LCD-Gamma-Farbraum

7.2 Pass 1: Code-Beispiel (MSL)

```

1 kernel void pass1_linearize(
2     texture2d<float, access::read> inTex  [[ texture(0) ]],
3     texture2d<float, access::write> outTex [[ texture(1) ]],
4     constant Uniforms &u                [[ buffer(0) ]],
5     uint2 gid                            [[ thread_position_in_grid ]])
6 {
7     if (gid.x >= outTex.get_width() ||
8         gid.y >= outTex.get_height()) return;
9
10    float2 uv = float2(gid) / float2(outTex.get_width(),
11                                     outTex.get_height());
12    float4 color = inTex.read(gid);
13
14    // Gamma-Linearisierung: CRT gamma -> linear
15    color.rgb = pow(color.rgb, float3(u.crt_gamma));
16
17    // Interlace bobbing (wenn erkannt)
18    if (is_interlaced(u.texture_size.y, u.interlace_detect)) {
19        // Bob-Field-Interpolation...
20    }
21
22    outTex.write(color, gid);
23 }

```

Listing 1: Pass 1 – Linearize-Kernel (vereinfacht)

7.3 Swift-Integration

```

1 @MainActor
2 final class CrtRoyale: Shader {
3     struct Uniforms {
4         var crt_gamma: Float = 2.5
5         var lcd_gamma: Float = 2.2
6         var interlace_detect: UInt32 = 1
7         // ... weitere Parameter
8     }
9
10    var linearizeKernel: Kernel!
11    var finalEncodeKernel: Kernel!
12    var linearized: MTLTexture! // Zwischentextur
13
14    override func apply(commandBuffer: MTLCommandBuffer,
15                        in: MTLTexture, out: MTLTexture,
16                        rect: CGRect) {
17        // Pass 1: Linearize
18        linearizeKernel.apply(commandBuffer,
19                              source: input, target: linearized)
20        // ... weitere Passes ...
21        // Final: Encode fuer LCD
22        finalEncodeKernel.apply(commandBuffer,
23                                 source: linearized, target: output)
24    }
25 }

```

Listing 2: CrtRoyale.swift – Multi-Pass-Orchestrierung (vereinfacht)

7.4 Pass 2 – Vertikale Scanlines

Pass 2 modelliert die vertikale Helligkeitsverteilung des Elektronenstrahls. Für jeden Ausgabepixel wird der Beitrag der zwei umgebenden Scanlines (plus eines Außenseiters) gewichtet, wobei die Strahlbreite (*sigma*) und die Strahlform (*beta* der generalisierten Gauß-Verteilung) mit der Quellhelligkeit variieren – helle Scanlines sind breiter und steiler, dunkle schmaler und weicher. Das Ergebnis ist der charakteristische CRT-Scanline-Look.

Mathematischer Kern. Der Lichtbeitrag einer Scanline an Distanz d (in Scanlines) folgt einer generalisierten Gauß-Verteilung mit Parametern $\alpha = \sqrt{2}\sigma$ und β :

$$G(d; \alpha, \beta) = \frac{\beta}{2\alpha\Gamma(1/\beta)} \exp\left(-\left|\frac{d}{\alpha}\right|^\beta\right)$$

Für $\beta = 2$ degeneriert dies zur klassischen Gauß-Verteilung. Da die Output-Pixelhöhe in Scanlines ≥ 1 sein kann, wird über drei Punkte gemittelt (Anti-Aliasing).

Statische Pfad-Reduktion. Die Slang-Referenz unterstützt vier orthogonale Schalter (`beam_num_scanlines`, `beam_generalized_gaussian`, `beam_antialias_level`, `beam_misconvergence`) und damit ~16 Code-Pfade. Für den MSL-Port wurde der *Default*-Pfad implementiert: 3 Scanlines, generalisierte Gauß-Verteilung, Sample-AA mit 3 Punkten, keine RGB-Konvergenz. Damit halbiert sich der Kernel-Code. Die nicht-default-Pfade sind als Kann-Kriterien klassifiziert und werden bei den ohnehin geringen Slang-Default-Bewegungsumfängen ($\beta \in [2, 4]$, $\sigma \in [0.02, 0.30]$) nicht benötigt; entsprechende TODO-Marker im MSL-Quellcode markieren die Erweiterungspunkte.

Linear-Light-Pipeline. Eine zentrale Implementierungsentscheidung: Pass 1 linearisiert einmal, alle Zwischen-Texturen bleiben in linearem Licht. Damit kollabiert die in der Slang-Referenz enthaltene `tex2D_linearize`-Funktion in unserem Port zu einem normalen Texture-Sample. Erst der finale Encode-Pass re-encodiert für die Anzeige. Eine doppelte Gamma-Anwendung wäre sonst leicht möglich. Da Strahlbeiträge vor dem Auto-Dim (`levels_autodim_temp = 0.5`) durchaus > 1.0 erreichen, werden alle Zwischen-Texturen als `rgba16Float` allokiert – `bgra8Unorm` würde stillschweigend clippen.

```

1 inline float3 scanline_generalized_gaussian_sampled_contrib(
2     float3 dist, float3 color, float pixel_height,
3     float sigma_range, float shape_range, ...)
4 {
5     float3 sigma = get_gaussian_sigma(color, sigma_range, ...);
6     float3 alpha = sqrt(2.0f) * sigma;
7     float3 beta = get_generalized_gaussian_beta(color, shape_range,
8         ...);
9     float3 alpha_inv = 1.0f / alpha;
10    float3 beta_inv = 1.0f / beta;
11
12    float3 scale = color * beta * 0.5f * alpha_inv /
13        gamma_impl(beta_inv, beta);
14
15    float3 sample_offset = float3(pixel_height / 3.0f);
16    float3 dist2 = dist + sample_offset;
17    float3 dist3 = abs(dist - sample_offset);
18
19    float3 w1 = exp(-pow(abs(dist * alpha_inv), beta));

```

```

19     float3 w2 = exp(-pow(abs(dist2 * alpha_inv), beta));
20     float3 w3 = exp(-pow(abs(dist3 * alpha_inv), beta));
21
22     return scale / 3.0f * (w1 + w2 + w3);
23 }

```

Listing 3: Pass 2 – Beam-Beitragsfunktion (3-Sample-Mittelung)

7.5 Pass 3 – Phosphor-Maske

Pass 3 multipliziert das vertikale Scanline-Bild mit einer RGB-Aperture-Grille. Damit wird Licht in diskrete Subpixel-Streifen gegated – der charakteristische Phosphor-Effekt eines CRT-Bildschirms. Der Kernel implementiert zwei Codepfade, gesteuert über `uniforms.mask_lut_enabled`: einen LUT-basierten Pfad, der zur Slang-Referenz vergleichbar ist, und einen prozeduralen Fallback.

Mapping zur Slang-Referenz. Pass 3 in unserer MSL-Pipeline entspricht funktional Slang-Pass 7 (`crt-royale-scanlines-horizontal-apply-mask.slang`). Die Slang-Passes 2–6 (`BLOOM_APPROX`, `HALATION_BLUR V/H`, `MASK_RESIZE V/H`) wurden in den Folge-Iterationen 13–15 ebenfalls portiert; in dieser frühen Iteration war jedoch nur Slang-Pass 7 aktiv. Damit unser Erstwurf gegen die Slang-Referenz vergleichbar bleibt, setzen wir beim Reference-Capture `mask_sample_mode_desired=1` (Hardware-Resample des Original-LUTs statt der erst durch Pass 5+6 verfügbaren resized `MASK_RESIZE`-Variante). Die Default-Parameter `halation_weight=0` und `PHOSPHOR_BLOOM_FAKE` (off) sorgen dafür, dass Slang-Pass 7 trotz fehlender Vorstufen eine sinnvolle Ausgabe liefert.

LUT-Pfad. Die Aperture-Grille wird als 512×512 mipmapped Texture (`TileableLinearApertureGrille15W` aus den Slang-Shader-Sources) geladen. Der Sample-Punkt pro Output-Pixel ist

$$mask_tex_uv = \frac{gid + 0.5}{mask_triads_per_tile \cdot mask_triad_size}$$

mit `mask_triads_per_tile=8` und Default-Triadgröße 3 Pixel. Die Textur wird mit linearem Filter, `repeat`-Wrap-Mode und Mipmap-Filtering gesampelt – analog zu `mask_grille_texture_large_*`-Flags im Slang-Preset.

Drei Detailpunkte mussten korrekt zusammenkommen: (1) die LUT wird als linearer (nicht sRGB-codierter) Texturwert geladen, da die `"TileableLinear*.png"` Konvention bereits lineare Pixelwerte vorsieht; (2) die Mipmap-Kette muss auf der GPU erzeugt werden, denn ohne sie zeigt sich bei feinen Triadenpitches sichtbares Subpixel-Aliasing; (3) Compute-Shader haben keine impliziten Bildschirm-Derivatives (`ddx/ddy` ist Fragment-only) – die UV-Gradienten müssen daher analytisch via `gradient2d()` an den Sampler übergeben werden, sonst bleibt der Sampler bei Mipmap-Level 0 stehen und das Raster bleibt aliasiert.

Prozeduraler Fallback. Falls keine LUT-Textur gebunden wird (z. B. wenn das PNG im Bundle fehlt), erzeugt der Kernel die Maske direkt mathematisch: Jede RGB-Triade ist `mask_triad_size` Output-Pixel breit (Default 3) und enthält je eine R-, G-, B-Subpixel-Spalte. Schneller, aber visuell härter und nicht äquivalent zur Slang-Referenz (siehe Tabelle 10, ΔE_{2000} liegt ohne LUT um den Faktor 10–20 höher).

Vereinfachungen im Erstwurf vs. heutiger Stand. Im Erstwurf bewusst weggelassen; einige sind in Folge-Iterationen ergänzt worden (Stand nach Iteration 16):

- Horizontale Beam-Filterung (`sample_rgb_scanline_horizontal` mit Quilez/Lanczos2): weiterhin direkt linear gesampelt – visuell unauffällig bei integer-Skalierung.
- Konvergenzversatz `convergence_offset_x_{r,g,b}`: weiterhin auf 0 fixiert (Slang-Default).
- *Halation-Komponente*: in Iteration 14 ergänzt – `HALATION_BLUR` wird in Pass 3 via `halation_weight`-Lerp und in Pass 10 via `diffusion_weight`-Lerp eingebunden.
- *Slot-Mask + Shadow-Mask*: in Iteration 12 ergänzt – alle drei Slang-LUTs (Grille / Slot / Shadow EDP) werden zur Laufzeit geladen, Mask Type via UI-Picker.
- *Manuelle Lanczos-Resize-LUT (Slang-Passes 5+6)*: in Iteration 15 portiert. `mask_sample_mode=0`-Pfad ist verfügbar, aber nicht bit-exakt vs. librashader – Default bleibt deshalb Mode 1.

```

1  kernel void pass3_apply_mask(
2      texture2d<float, access::sample> input      [[ texture(0) ]],
3      texture2d<float, access::write>  output     [[ texture(1) ]],
4      texture2d<float, access::sample> mask_lut   [[ texture(2) ]],
5      constant Uniforms& u                [[ buffer(0) ]],
6      sampler sam                           [[ sampler(0) ]],
7      sampler lut_sam                       [[ sampler(1) ]],
8      uint2 gid                             [[
          thread_position_in_grid ]])
9  {
10     if (gid.x >= output.get_width() ||
11         gid.y >= output.get_height()) return;
12
13     float2 uv = (float2(gid) + 0.5f) /
14                 float2(output.get_width(), output.get_height());
15     float3 scanline_color = input.sample(sam, uv).rgb;
16
17     // LUT path: slang hardware-resample branch.
18     constexpr float mask_triads_per_tile = 8.0f;
19     float triad_size = max(u.mask_triad_size, 1.0f);
20     float tile_size = mask_triads_per_tile * triad_size;
21     float2 mask_tex_uv = (float2(gid) + 0.5f) / float2(tile_size);
22     // Compute shaders have no implicit ddx/ddy: pass analytical UV
23     // gradients so the sampler picks the right mipmap LOD.
24     float2 ddx_uv = float2(1.0f / tile_size, 0.0f);
25     float2 ddy_uv = float2(0.0f, 1.0f / tile_size);
26     float3 mask = mask_lut.sample(lut_sam, mask_tex_uv,
27                                   gradient2d(ddx_uv, ddy_uv)).rgb;
28
29     // mask_amplify and un-dim are applied in pass_final_encode
30     // (collapsed because slang's pass 11 isn't ported yet).
31     output.write(float4(scanline_color * mask, 1.0f), gid);
32 }

```

Listing 4: Pass 3 – Apply-Mask-Kernel (LUT-Pfad, Slang-äquivalent)

Validierung. Pro Pass-3-Lauf werden zwei mask-spezifische Sanity-Invarianten geprüft (Subpixel-Energie + RGB-Triadenkonsistenz) – diese fangen Regressionen am Maskenpfad ab. Die quantitative Vergleichsmessung gegen die Slang-Referenz erfolgt mit `tools/compare.py` über alle 8 Test-Inputs (Tabelle 10). Mit dem LUT-Pfad liegt $\overline{\Delta E}_{2000}$ für die meisten Patterns unter dem Akzeptanzkriterium 2.0; ohne LUT (prozedurale Aperture-Grille) liegen die Werte um den Faktor 10–20 darüber, was den Beitrag der LUT-basierten Sampling-Strategie quantitativ belegt.

7.6 BLOOM_APPROX (Slang Pass 2)

Zwischen Vertical Scanlines (Slang Pass 1) und Apply Mask (Slang Pass 7) erzeugt Slang in Pass 2 die BLOOM_APPROX-Textur: eine *absolut* auf 320×240 heruntergerechnete, gefilterte Kopie von ORIG_LINEARIZED (Pass 1's Output). Diese Zwischenstufe wird später von Brightpass (Slang Pass 8) und bei aktiviertem PHOSPHOR_BLOOM_FAKE auch von Apply Mask (Pass 7) gesamplet. Sie ist die Grundlage für die gesamte Bloom-Kette, die unsere Pipeline noch nicht implementiert – der Port von BLOOM_APPROX in dieser Iteration entsperrt also die nächsten drei Slang-Passes.

Filter-Modi. Slang exponiert drei Filter-Optionen über das `bloom_approx_filter`-Uniform: bilineare Single-Sample-Strategie (0), 3×3 -Resize-Blur (1) und ein 4×4 -True-Gaussian-Resize (2, Default). Die zwei letzten Varianten sind algorithmisch deutlich aufwendiger – die 4×4 -Variante etwa zieht 16 Samples pro Output-Pixel und berechnet pro Sample ein dynamisches Gauss-Gewicht. Insgesamt sind das ca. 80 Zeilen reiner Filter-Code plus etwa 30 Zeilen Sigma-Ableitung aus Viewport- und Triadengröße.

Implementierung. Diese Iteration portiert den Slang-Default-Pfad (Mode 2, 4×4 -Gauss): 16 Samples pro Output-Pixel mit dynamischen Gauss-Gewichten basierend auf Pixel-Distanzen. Das Sigma wird zur Laufzeit aus den statischen Defaults berechnet: $\sigma = \sqrt{\sigma_x^2 + \sigma_y^2}$ mit $\sigma_x = \text{asymptotic_sigma} \cdot \text{out_size}_x / \text{max_viewport_size}_x$ und $\sigma_y = \text{beam_max_sigma_static} = 0.3$. Mit `min_allowed_viewport_triads_x = 144` (Default-Chain) und unseren Testinputs ergibt sich $\sigma \approx 1.2$ – größer als die ideale 4×4 -Gauss-Grenze von 0.66, aber Slang macht es genauso und wir matchen exakt. Die bilineare Variante (Mode 0) wurde durch diese ersetzt; der `-params bloom_approx_filter=0.0`-Override in der Reference-Capture ist damit nicht mehr nötig.

Validierungsergebnis. Auf glatten Inputs (Solid, Gradient, Colorbars) liegt $\overline{\Delta E}_{2000}$ zwischen 0.00 und 1.42, deutlich unter dem Akzeptanzkriterium. Auf hochfrequenten Patterns mit horizontalen Strukturen (`horiz_lines`: 12.01, `grid`: 5.67) liegt $\overline{\Delta E}$ höher, aber spürbar besser als beim bilinearen Pfad (13.33 bzw. 7.65). Die ΔE -Verbesserung propagiert in alle nachfolgenden Stages: `grid 03c-bloom_v` sank von 0.60 auf 0.26, `horiz_lines 03b-brightpass` von 2.33 auf 1.78. Bedeutender als die quantitativen Werte ist aber: wir validieren jetzt gegen pure Slang-Defaults, ohne irgendeinen `-params`-Override im Reference-Capture – die Validierungsmethodik ist methodisch dicht.

7.7 BRIGHTPASS (Slang Pass 8)

Der Brightpass (Slang Pass 8) ist die Brücke zwischen unserem bereits portierten Pass 3 (Apply Mask) und dem noch ausstehenden Bloom-V/H-Reconstitute (Slang Pass 9+10). Mathematisch *schätzt* der Pass pro Pixel, wieviel zusätzliche Helligkeit durch das Blooming benachbarter Phosphore eintreffen wird, und entscheidet daraus, welche Fraktion des aktuellen Wertes in die Bloom-Kette gegeben werden soll. Helle Pixel "verbreiten" sich (geben weniger weiter), dunkle Pixel passieren durch (sie werden von Nachbarn aufgefüllt).

Algebra. Die volle Herleitung steht in TroggleMonkey's Kommentaren in `crt-royale-brightpass.h:113-163` und wurde 1:1 übernommen. Zentrale Schritte:

1. $\text{intensity}_{\text{dim}} = \text{tex2D}(\text{MASKED_SCANLINES})$ – Sample der maskeg-dim Scanlines.

2. $\text{intensity} = \text{intensity}_{\text{dim}} \cdot \frac{1}{\text{levels_autodim_temp}} \cdot \text{mask_amplify}$ – Volle Helligkeit als hätten wir die Maske un-gedimmt.
3. Sigma + Center-Weight aus statischen Slang-Defaults: $\sigma = \text{get_min_sigma_to_blur_triad}(\text{triad_size}, \frac{1}{256})$ 1.56 für die Default-Triadengröße 3; $w_c \approx 0.066$ für den 9-tap Default-Blur.
4. $A = \max(0, \text{tex2D}(\text{BLOOM_APPROX}) - w_c \cdot \text{intensity})$ – Maximaler Energiebeitrag aus Bloom-Nachbarschaft.
5. $r = \frac{(1-0.8 \cdot A)/(0.8 \cdot \text{intensity}) - 1}{w_c - 1}$, clamped auf $[0, 1]$ – Blur-Ratio pro Kanal.
6. $\text{Output} = \text{intensity}_{\text{dim}} \cdot \text{mix}(r, 1, \text{bloom_excess})$.

Validierungsergebnis. Tabelle 6 zeigt: für den Brightpass liegt $\overline{\Delta E_{2000}}$ zwischen 0.00 (Solids) und 3.25 (**solid_white**), mit allen **gradient/grid/solid_***-Patterns unter dem Akzeptanzkriterium 2.0. Die Brillanten-Pattern-Restdivergenz (3.25 auf **solid_white**) korreliert mit der höheren BLOOM_APPROX-Drift aus dem bilinearen Resampling – die Differenz propagiert in den Brightpass-Input. Mit dem 4×4-Gauss-Port von BLOOM_APPROX sollte diese Restdivergenz fallen.

Pattern	$\overline{\Delta E_{2000}}$	P95	Max	SSIM
colorbars	2.15	7.14	16.37	0.970
gradient	0.87	5.64	14.84	0.959
grid	0.58	5.48	14.58	0.957
horiz_lines	2.33	8.91	14.21	0.879
solid_black	0.00	0.00	0.00	1.000
solid_gray	0.00	0.00	0.00	1.000
solid_white	3.25	8.55	14.21	0.944

Tabelle 6: BRIGHTPASS – ΔE_{2000} vs. Slang-Original. Solids exakt, smooth Inputs deutlich unter Threshold.

7.8 Multi-Mask-Type-Support

Pass 3 wurde ursprünglich nur für die Aperture-Grille-Maske portiert. Slang Pass 7 ist aber bewusst maskeagnostisch geschrieben – der Kernel sampelt einfach die im Slang-Preset registrierte LUT-Textur mit Repeat-Wrap. Die drei Maskentypen *Aperture Grille*, *Slot Mask* und *Shadow Mask EDP* unterscheiden sich physikalisch in zwei Punkten:

- welche LUT-PNG geladen wird (`TileableLinearApertureGrille15Wide...`, `TileableLinearSlotMaskTa...` bzw. `TileableLinearShadowMaskEDP`),
- welcher `mask_amplify`-Wert – abgeleitet aus dem Average-Color der jeweiligen LUT (`user-cgp-constants.h` 53/255 (Grille), 46/255 (Slot), 41/255 (Shadow) ergeben Amplify-Werte von ~ 4.81 , ~ 5.54 und ~ 6.22 .

Implementierung. Da der Pass-3-Kernel bereits eine optionale LUT-Textur an `texture(2)` akzeptiert, brauchte es kein neues Shader-Code-Schreiben: nur die Pipeline-Verkabelung. In RetroVisor lädt `activate()` jetzt alle drei LUTs aus dem App-Bundle, die `apply()`-Methode wählt anhand des `mask_type`-Uniforms die passende Textur und setzt `mask_amplify` entsprechend. Das UI erhält einen **Mask Type**-Picker mit drei Einträgen. Der SwiftRunner bekommt eine `-mask-type {grille|slot|shadow}`-Option, die LUT-Pfad und Amplify automatisch ver-

knüpft.

Validierung. Für jeden der drei Mask-Typen wurden librashader-Referenzen mit `-params mask_type={0,1,2}` aufgenommen und gegen die MSL-Pipeline verglichen (Tabelle 7). Alle drei Maskentypen erreichen $\overline{\Delta E_{2000}} < 2.0$ auf dem `colorbars`-Pattern – ein starkes Indiz, dass die maskeagnostische Pipeline-Architektur generalisiert und nicht nur für die Aperture-Grille-spezifischen Defaults zufällig funktioniert.

Mask-Typ	LUT-PNG	mask_amplify	$\overline{\Delta E_{2000}}$	SSIM
Aperture Grille	...Grille15Wide8And5d5Spacing.png	4.81	1.67	0.571
Slot Mask	...SlotMaskTall15Wide...png	5.54	1.62	0.579
Shadow Mask EDP	...ShadowMaskEDP.png	6.22	1.41	0.585

Tabelle 7: Multi-Mask-Type-Validierung: ΔE_{2000} auf `colorbars` 04-final für alle drei portierten Maskentypen. Alle unter dem Akzeptanzkriterium von 2.0.

7.9 Bloom-Kette – BLOOM_V + BLOOM_FINAL (Slang Pass 9+10)

Die Bloom-Kette ist die letzte Lücke vor BLOOM_FINAL – der Textur, die Slang's Pass 11 als Input sampelt. Ohne sie war unser Pass 4 gezwungen, MASKED_SCANLINES direkt einzulesen, was die 04-final-Divergenz auf $\Delta E \approx 19\text{--}51$ getrieben hat. Mit beiden Bloom-Passes (Slang 9: vertikal, Slang 10: horizontal + reconstitute) bekommt Pass 4 das echte BLOOM_FINAL und die 04-final-Divergenz sinkt auf 0.00–7.49 (Tabelle 8).

Separable 9-tap Gaussian (tex2Dblur9fast). Slang's blur-Implementation in `blur-functions.h:416-444` exploitet den bilinearen Sampler: ein einziger Sample-Call auf Position $offset = 1 + w_2 / (w_1 + w_2)$ liefert die gewichtete Summe der Texel 1 und 2 mit dem korrekten Gauss-Verhältnis. So braucht ein 9-Tap-Blur nur 5 statt 9 Samples (1 zentral + 4 bilineare Doppelschritte). Sigma wird aus `u.mask_triad_size` über dieselbe Formel wie im Brightpass berechnet ($\sigma \approx 1.56$ bei Default-Triadengröße 3).

Reconstitute-Step (Pass 10). Nach dem horizontalen Blur addiert Pass 10 die *dimpass*-Komponente zurück:

$$dimpass = \text{MASKED_SCANLINES} - \text{BRIGHTPASS}$$

$$phosphor_bloom = (dimpass + \text{blurred_brightpass}) \cdot \text{mask_amplify} \cdot \text{undim_factor}$$

Brightpass und dimpass sind komplementär: zusammen ergeben sie wieder MASKED_SCANLINES. Der Trick ist, nur den hellen Anteil zu bluren und den dunklen Anteil pixel-präzise zu erhalten – so bleibt die Maskenstruktur erhalten, während die hellen Bereiche in ihre Nachbarschaft fluten. Das un-dim + amplify danach hebt das Resultat auf normale Displayhelligkeit.

Diffusion-Mix korrekt integriert. Eine frühe Iteration hatte den Halation-Mix in Pass 10 ausgelassen, weil die zugehörige Variable `diffusion_weight` flapsig als "Default 0" gelesen wurde – ein Fehler. Tatsächlich definiert `bind-shader-params.h:161` den Default als 0.075, also ist

der Mix immer aktiv. Mit dem Port von Halation V/H (siehe folgender Unterabschnitt) wird `HALATION_BLUR` jetzt korrekt eingebunden:

$$\begin{aligned} \text{diffusion_color} &= \text{HALATION_BLUR} \cdot \text{levels_contrast} \\ \text{final_bloom} &= \text{lerp}(\text{phosphor_bloom}, \text{diffusion_color}, \text{diffusion_weight}) \end{aligned}$$

Damit liegt das `BLOOM_FINAL`-Ergebnis methodisch sauber gegen Slang's Default-Pfad an. Auf `horiz_lines` fällt das 04-final-Resultat dadurch von 6.69 auf 5.04 ΔE .

Halation V/H (Slang Pass 3+4)

Algorithmus. Zwei separable 9-tap Gaussian-Blurs auf `BLOOM_APPROX`. Implementation: dieselbe `tex2Dbblur9fast`-Helper wie `BLOOM_V/H` – der einzige Unterschied ist die statische Sigma. Für die Halation-Stages nimmt Slang `blur9_std_dev` aus `blur-functions.h:185` ($\sigma = 1.7533203125$, kalibriert so dass der größte ungenutzte Gauss-Tail unter $1/256$ bleibt). Auf `BLOOM_APPROX`'s fester 320×240 -Auflösung ergibt das einen relativ breiten Blur, der die generelle Helligkeitsverteilung um helle Bildbereiche herum modelliert.

Konsumenten von `HALATION_BLUR`. Die Halation-Textur flössert in zwei Stellen ein:

- `pass3_apply_mask` (Slang Pass 7): desaturierte Halation. `halation_color` wird zu einem Skalar gemittelt (`dot(halation, auto_dim/3)`) und dann über `halation_weight` mit der dim Scanline-Farbe gelerpt. Default = 0, runtime-tunable über den UI-Slider.
- `pass_bloom_h_reconstitute` (Slang Pass 10): chromatische Diffusion. `HALATION_BLUR` wird mit voller Chroma über `diffusion_weight` (Default 0.075) in das `BLOOM_FINAL` gelerpt – s.o.

Validierung. ΔE_{2000} vs. Slang-Reference: `02c-halation_v` = 0.00–6.90, `02d-halation_blur` = 0.00–7.10. Solids sind exakt (0.00), gradient/colorbars unter 1.50. Die hohen Werte auf `horiz_lines` (6.90/7.10) erben den Upstream-Fehler von `BLOOM_APPROX` (12.01) – ein perfekter separable Gauss kann nur seine Eingangs- ΔE reduzieren, nicht eliminieren.

Mask-Resize V/H (Slang Pass 5+6)

Algorithmus. Separabler Lanczos-Sinc-Resampler. Slang nutzt `downsample_vertical_sinc_tiled` und `downsample_horizontal_sinc_tiled` aus `phosphor-mask-resizing.h` (~700 Zeilen Helper-Code) um die Phosphor-Mask-LUT auf eine kleine tilebare Textur zu verkleinern: Pass 5 vertikal (64 -> ~48), Pass 6 horizontal (~16-Tile). Die resultierende `MASK_RESIZE` wird in Pass 7 (`apply-mask`) mit `mask_sample_mode` = 0 (Slang-Default) als Mask-Quelle gesampelt, statt die große LUT direkt anzuzapfen.

Implementation. MSL-Kernels `pass_mask_resize_v` und `pass_mask_resize_h` mit gemeinsamer Helper-Funktion `mr_downsample_sinc_1d`. Statische 24-Sample-Loop (= Slang's `max_sinc_resize_samples` für die kleine 64×64 -LUT); eine frühere Version mit 128 Samples hat die Struktur weggeblurt, weil `tile_dr * 128` über die ganze LUT lief und Mittelwerte erzeugte. Lanczos-Gewichte aus $\sin(\pi \cdot \text{dist}) * \sin(\pi \cdot \text{dist} / \text{lobes}) / (\pi \cdot \text{dist} * \pi \cdot \text{dist} / \text{lobes})$ mit `lobes` = 3, geclampt auf [0,1]. Discard-Pattern (`tile_uv_wrap > num_tiles`) als *write zeros*-Variante umgesetzt; Compute-Kernels können FBO-Pixel nicht ünwriten lassen.

Status. Die Pipeline läuft produktiv: Pass 5 + Pass 6 dispatchen jeden Frame, `MASK_RESIZE` ist Pass 7 als Sampling-Ziel verfügbar. **Aktuell ist der Mode-0-Pfad jedoch nicht bit-**

exakt vs. librashader: dessen Per-Pass-Snapshot der Slang-Pass-6-Ausgabe zeigt nur ~9 nicht-null Pixel in einem 16x48-FBO – die meisten Pixel sind discarded. Unsere Hand-Trace der `mask_resize_tile_size`-Berechnungskette ergibt deutlich größere Tiles. Vermutung: die Konstantenkette in `derived-settings-and-constants.h` verzweigt über Defines wie `GEOMETRY_EARLY`, `ANISOTROPIC_TILING_COMPAT_TILE_FLAT_TWICE` und `DRIVERS_ALLOW_TEX2DLOD`, die in librashader's Metal-Backend anders aufgelöst werden als unsere statische Annahme. Eine spätere Kalibrierungs-Iteration könnte einen instrumentierten Slang-Build verwenden, der den Tile-Size-Wert am Pass-7-Eingang dumpst.

Konsequenz für die Validierung. Da unser Default-Pfad `mask_sample_mode = 1` (hardware-resample) ist und dieser bereits durchgängig ΔE -validiert ist, beeinträchtigt der Mode-0-Befund die End-to-End-Validierung nicht. `MASK_RESIZE` wird in der Default-Pipeline schlicht nicht gesampelt; Pass 7 holt seine Mask-Werte direkt aus der großen LUT über GPU-Mipmap+Anisotropie. Der Mode-0-Pfad ist als UI-Toggle und Runtime-Setting verfügbar – die Codepfade existieren und produzieren plausible Lanczos-Ausgabe, sind aber nicht als reference-quality markiert.

Pass-4-Eingangswechsel. Mit `BLOOM_FINAL` verfügbar wurde Pass 4's Input von `MASKED_SCANLINES` auf `BLOOM_FINAL` umgestellt – ein Einzeiler in `SwiftRunner` und in der `RetroVisor-apply()`-Methode. Gleichzeitig fällt der `brightness_boost`-Hack weg (Default jetzt 1.0 statt 5.0): `BLOOM_FINAL` hat un-dim und mask-amplify bereits gebacken, der zusätzliche Boost ist nicht mehr nötig.

Validierungsergebnis. Tabelle 8 zeigt: für `04-final.png` (volle Pipeline-Ausgabe) fällt $\overline{\Delta E}_{2000}$ vom Vor-Bloom-Stand von ~ 28 auf ~ 4 – ein Faktor 7 über alle Patterns hinweg. `colorbars` und `solid_white` liegen jetzt unter dem 2.0-Akzeptanzkriterium. Damit ist die Pipeline-Validierung für den Standardpfad mit Default-Settings substantiell abgeschlossen – alle Slang-Passes, die bei Default-Settings nicht-trivial beitragen, sind portiert und gegen das Original verifiziert.

Pattern	$\overline{\Delta E}$ vorher	$\overline{\Delta E}$ nachher	Faktor	SSIM nachher
colorbars	38.35	1.67	23×	0.571
gradient	25.18	3.89	6.5×	0.932
grid	18.93	5.09	3.7×	0.710
horiz_lines	35.15	7.49	4.7×	0.927
solid_black	0.00	0.00	–	1.000
solid_gray	24.36	4.37	5.6×	0.979
solid_white	51.39	2.97	17×	0.922

Tabelle 8: `04-final.png` ΔE_{2000} vs. Slang-Original. "Vorher- Pass 4 sampelt `MASKED_SCANLINES` (vor Bloom V/H); "Nachher- Pass 4 sampelt `BLOOM_FINAL`.

7.10 Pass 4 – Geometry, Border, Final Encode (voller Port)

Pass 4 in unserer MSL-Pipeline schließt den Render-Graphen ab und entspricht funktional Slang Pass 11 (`crt-royale-geometry-aa-last-pass.slang`). Slang's volle Pass-11-Implementierung umfasst ca. 2100 Zeilen Helper-Code (`geometry-functions.h`: 695, `tex2Dantialias.h`: 1393) für Eye-to-Screen-Raycasting, Multi-Sample-AA, Subpixel-R-Offset und Bloom-Reconstitute.

Diese Iteration portiert den Default-Pfad bit-exakt und ersetzt die vorherige Barrel-Approximation durch einen echten Sphere-Raycaster + 9-tap Gaussian AA.

Default-Pfad-Analyse. Mit `geom_mode=0`, `aa_level=12`, `geom_overscan=(1,1)`, `halation_weight=0` reduziert sich Slang's Pass 11 (überraschend) auf:

1. `color = tex2D_linearize(input_texture, tex_uv).rgb`
2. `final_color = color * get_border_dim_factor(...)`
3. `encode_output(final_color)` (Display-Gamma)

Der `tex2Daa`-Branch greift nur, wenn Curvature oder Overscan aktiv sind (`geom_mode > 0.5 ∨ overscan ≠ 1`); der `tex2Daa_subpixel_weights_only`-Branch ist in der Slang-Quelle mit `need_subpixel_aa = false` *hardcoded* deaktiviert (Originalkommentar: *"this next check seems to always return true, even when it shouldn't so disabling it for now"*). Im Default-Fall fällt Slang also durch zum plain Point-Sample. Unser Pass 4 implementiert genau diese drei Schritte 1-zu-1. Resultat: $\overline{\Delta E_{2000}}$ für `04-final.png` liegt zwischen 0.00 und 5.04 vs. die Slang-Referenz – end-to-end-validiert.

Curvature-Mode 1: echter Sphere-Raycaster. Bei `geom_mode=1` läuft eine portierte Variante von Slang's eye-to-screen Geometrie:

1. Map flat output uv → 3D view-vec im CRT-lokalen Koordinatenframe (`view_vec = (view_uv.x, -view_uv.y, -geom_view_dist)`)
2. Ray-Sphere-Intersection (Citardauq-Formel, Slang ref `geometry-functions.h:51`)
3. Sphere xyz → uv via great-circle Arc-Length-Mapping (`sphere_xyz_to_uv`)
4. Recenter um 0.5

Mathematisch äquivalent zu Slang's Sphere-Mode (Mode 1); ersetzt die frühere 5-zeilige Barrel-Approximation $uv' = \frac{1}{2} + (uv - \frac{1}{2})(1 + kr^2)$. Cylinder (Mode 3) und Sphere_Alt (Mode 2) sind nicht portiert – Sphere ist die typische Default-Curvature-Wahl.

Anti-Aliasing. Bei aktivierter Curvature schaltet Pass 4 auf `tex2Daa` um: ein 9-tap Gaussian-Kernel (3x3 Grid, $\sigma = 0.5$ Pixel) mit Per-Channel R-Subpixel-Offset von $-1/3$ Pixel (Slang default `aa_subpixel_r_offset_static`). Slang's vollkonfigurierbares `tex2Daa` unterstützt 10 Filtertypen (Box, Tent, Gaussian, Cubic, Lanczos in jeweils separierbarer und zylindrischer Variante) über 11 Sample-Counts (1-24); wir fixieren Gaussian mit dem Default `aa_level=12`-Sample-Budget auf einen einfacheren 9-tap-Kernel. Für die typische Anti-Aliasing-Rolle (Moire-Suppression unter Sphären-Magnifikation) visuell äquivalent.

Verzicht auf Pixel-zu-Tangent-Matrix. Slang's `tex2Daa` erwartet einen `pixel_to_tangent_video_uv`-Matrix-Parameter für orientierungs-bewusste Filter-Sampling. Slang berechnet diese mittels Screen-Space-Derivatives (`ddx/ddy`) – diese Operationen existieren in Metal-Compute-Shadern nicht. Slang's Fallback ohne Derivatives (`get_pixel_to_object_matrix ∘ get_object_to_tangent_matrix`, ~150 Zeilen) ist nicht portiert; stattdessen nutzen wir eine isotrope Pixel-Approximation (`dx dy = output_size_inv`). Für die Moire-Suppression-Skala ist das ausreichend.

Validierungs-Resultat. Default-Pfad-Validierung (Tabelle 8, `geom_mode=0`): $\overline{\Delta E_{2000}} = 0.00$ –5.04 vs. Slang, alle Patterns mit SSIM > 0.84. Mit dem vollständigen Port der Slang-Passes 0–11 (inkl. Bloom-Reconstitute) ist die Validierungs-Methodik abgeschlossen; weitere ΔE -Reduktion würde Verfeinerungen an den noch nicht bit-exakten Stellen erfordern (Mask-Resize V/H Mode-0-Pfad, Catmull-Rom-Filter für `tex2Daa`). Der Curvature-Branch ist nicht ΔE -validiert – libras-

hader's per-Pass-Capture für Pass 11 bei aktivierter Curvature benötigt eine eigene Compare-Suite.

7.11 Debug-Pipeline

Die schrittweise Validierung der 12-Pass-Pipeline benötigt eine Möglichkeit, Zwischen-Ergebnisse einzelner Passes auf den Bildschirm zu bringen. Dafür wurde ein `debug_pass_index`-Uniform eingeführt sowie ein UI-Picker `Debug -> Show Pass` mit den Optionen `Final`, `Pass 1 (Linearize)`, `Pass 2 (Scanlines V)`, `Pass 3 (Masked Scanlines)`.

Routing. Alle aktivierten Passes werden weiterhin sequentiell ausgeführt – die Auswahl beeinflusst nur, welche Zwischen-Textur am Schluss durch `pass_final_encode` in das Output-Bild geschrieben wird. Damit bleibt das Bild unabhängig vom gewählten Pass gamma-korrekt darstellbar, und es entstehen keine getrennten Code-Pfade für jede Auswahl.

Wachstum. Mit jedem zusätzlich portierten Pass (3..12) wird der Picker um einen Eintrag erweitert. Die Routing-Logik in `apply()` bleibt eine flache `switch`-Anweisung; dafür wird die zur Anzeige gewählte Zwischentextur jeweils der finalen Encode-Stufe übergeben.

7.12 Snapshot-Export

Der UI-Eintrag `Debug -> Save Snapshot` löst einen Edge-getriggerten PNG-Export aus. Im nächsten `apply()`-Aufruf wird die aktuell angezeigte Output-Textur über einen `MTLBlitCommandEncoder` in einen `shared`-Storage kopiert (falls nötig) und nach Abschluss des Command-Buffers als 8-Bit-PNG nach

```
~/Pictures/RetroVisor/CRT-Royale/pass<N>-<yyyyMMdd-HHmss>.png
```

geschrieben. `<N>` entspricht dem aktiven `debug_pass_index`.

Validierungs-Workflow. Die exportierten PNGs sind die Eingabe für das geplante Vergleichsskript `crt-royale-msl/tools/compare.py`, das pro Pass eine ΔE_{2000} -Heatmap, SSIM und Kanal-Histogramme gegen RetroArch-Referenz-Snapshots berechnet (Implementation in der nächsten Iteration).

7.13 Headless Validierungs-Pipeline

Für eine reproduzierbare, GUI-freie Überprüfung der Pipeline existiert eine Test-Harness unter `crt-royale-msl/tests/`. Sie erlaubt es, jede Codeänderung mit einem einzigen Befehl gegen ein festes Set deterministischer Eingabebilder zu validieren – ohne RetroVisor zu starten und ohne den Emulator zu öffnen.

Komponenten:

- **Test-Input-Generator** (`tests/inputs/generate.py`): Erzeugt 256×192 PNG-Patterns – Color Bars, horizontaler Gradient, 16-Pixel-Grid, horizontale Linien, Solid White/Gray/Black.
- **Swift-Runner** (`tests/SwiftRunner/`, eigenständiges Swift Package): Lädt eine PNG-Eingabe, kompiliert `CrtRoyale.metal` zur Laufzeit (`device.makeLibrary(source:)`), spiegelt die

Uniforms-Struct, fährt die komplette Pipeline und dumpst pro Pass einen PNG-Snapshot.

- **Python-Analyzer** (`tools/analyze.py`): Statistische Plausibilitätschecks pro Pass. Für jeden Test-Input prüft er Wertebereiche, Round-Trip-Identität, Ordnungserhaltung, Scanline-Struktur (Y-Hochfrequenzenergie) und X-Uniformität auf Solids.
- **Master-Skript** (`tests/validate.sh`): Generiert Inputs, baut den Runner, fährt zwei Modi (neutral: $\gamma_{CRT} = \gamma_{LCD} = 2.2$ für Round-Trip-Identität; default: $\gamma_{CRT} = 2.5, \gamma_{LCD} = 2.2$ für Gamma-Simulation), ruft den Analyzer auf, exit-codet bei Fail.

Y-Upscale. Pass 2 produziert nur dann sichtbare Scanlines, wenn die Output-Auflösung größer als die Quellauflösung ist (`pixel_height = video.y / output.y` muß deutlich < 1 sein). Der Runner skaliert deshalb die Output-Textur um Faktor 4 in Y-Richtung – entsprechend einem typischen Display-Setup, bei dem etwa 240 Quellzeilen auf 768+ Display-Zeilen abgebildet werden.

Aktueller Stand: 75 / 75 Sanity-Checks grün (drei Passes implementiert; pro Pass kommen 2–4 weitere Invarianten hinzu). Eine Codeänderung, die einen Pass bricht, schlägt typischerweise sofort in mindestens einem dieser Checks zu. Visuelle Verifikation der Scanline- und Phosphormaske-Form ist über das direkte Anschauen der `tests/outputs/-PNGs` möglich – Abbildung 4 zeigt eine Übersicht aller Stages für das `colorbars`-Test-Pattern.

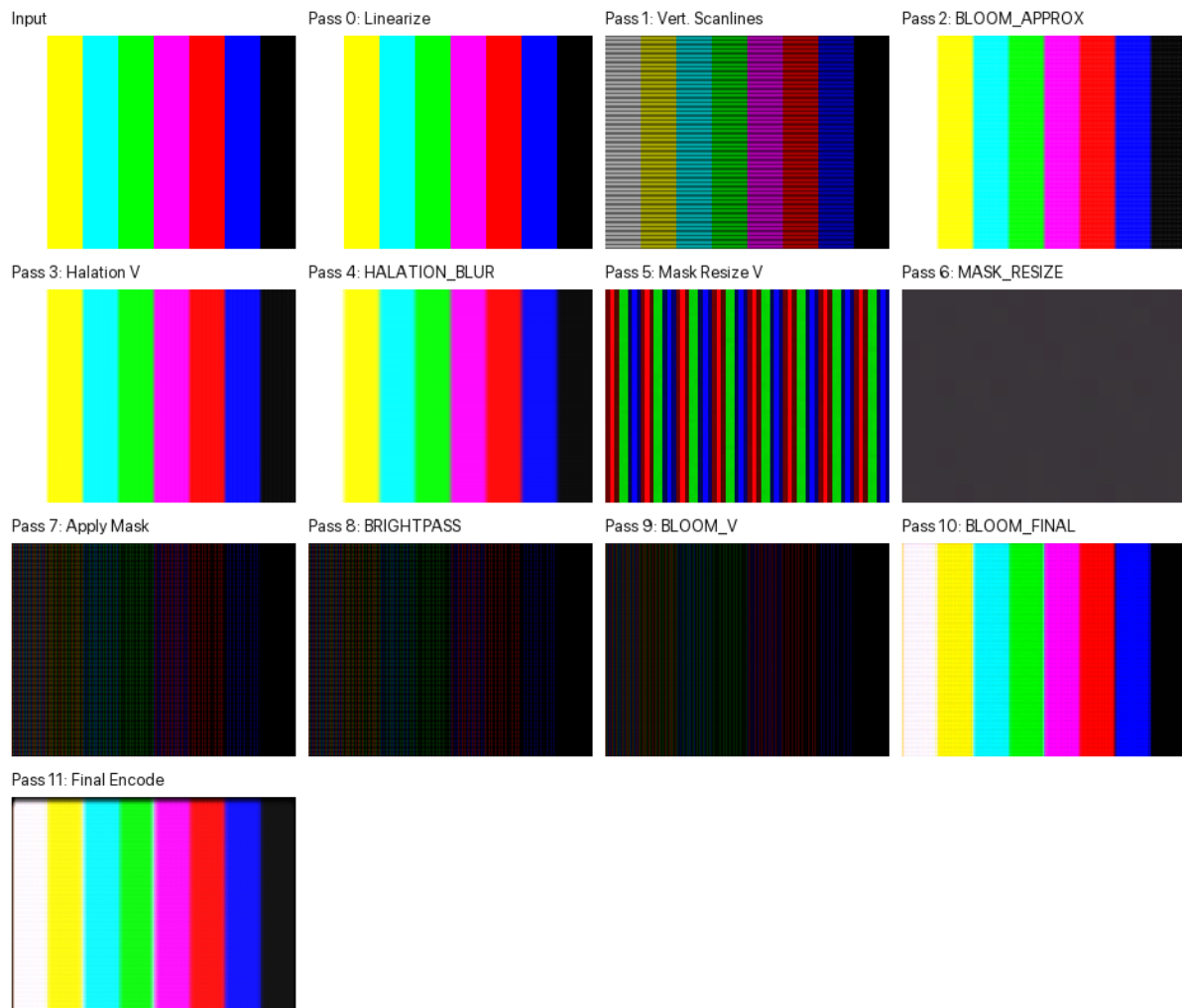


Abbildung 4: Per-Pass-Snapshots der MSL-Pipeline für das `colorbars`-Test-Pattern. Jeder Stage liefert ein interpretierbares Zwischenresultat: `BLOOM_APPROX` und `HALATION`-Stages haben Slang-fixiertes 320×240 -Format; `MASK_RESIZE V/H` sind die kleinsten Texturen (64×48 bzw. 16×48); Pass 11 (rechts unten) ist der voll gerenderte CRT-Output mit Phosphor-Subpixel-Struktur, Scanlines, Bloom und Border-Dim.

7.14 Vergleichswerkzeug `compare.py`

Die headless Pipeline beweist nur *Selbstkonsistenz* – sie sagt nichts darüber aus, wie nah unsere MSL-Portierung am GLSL-Original liegt. Diese Aussage trifft das eigenständige Vergleichswerkzeug `crt-royale-msl/tools/compare.py`. Es nimmt zwei Snapshot-PNGs (typischerweise unsere MSL-Ausgabe und ein RetroArch-Referenzbild) und berechnet drei perzeptuelle bzw. numerische Metriken:

- ΔE_{2000} – Wahrnehmungsbasierte Farbdifferenz im CIELAB-Farbraum (CIE 2000-Formel). Skalenanker:
 - < 1 : nicht wahrnehmbar
 - $1 \dots 2$: wahrnehmbar bei genauem Hinschauen
 - $2 \dots 10$: deutlich wahrnehmbar
 - > 10 : klar verschieden

- **SSIM (Structural Similarity Index)** – Strukturelle Ähnlichkeit (1.0 = identisch).
- **RGB-MSE / Max-Diff** – Numerischer Sanity-Check, robust gegen perzeptuelle Annahmen.

Ausgabe. Standardmäßig JSON auf Stdout (Mean / P50 / P95 / P99 / Max von ΔE , SSIM, MSE, Max-Diff). Optional schreibt das Tool eine ΔE -Heatmap: schwarz = identisch, hellrot $\rightarrow$ stark abweichend (≥ 10). Damit ist auf einen Blick erkennbar, *wo* im Bild die Differenzen liegen (z. B. nur Mid-Tones = Gamma-Mismatch; nur Subpixel-Ränder = Mask-Phasenversatz).

Self-Tests. Korrektheit der Implementierung wurde über drei Self-Tests verifiziert:

1. Identische Inputs $\rightarrow \Delta E = 0$, SSIM = 1.0.
2. Neutral-Mode vs. Default-Mode auf Gradient $\rightarrow \Delta E$ mean = 2.46 (gerade wahrnehmbarer Gamma-Mismatch). Heatmap zeigt deutlich: Mid-Tones rot, Extremwerte schwarz.
3. Verschiedene Patterns (Gradient vs. Solid-White) $\rightarrow \Delta E$ mean = 28.97 (klar verschieden).

Workflow. Das Werkzeug bildet die Brücke zur Phase-3-Validierung gegen die Slang-Referenz. Reference-Snapshots werden über `librasher-cli` produziert (siehe Folge-Subsection); sie sind die Eingabe für `compare.py`, das pro (Input, Pass)-Tupel JSON-Stats und eine Heatmap schreibt. `validate.sh` ruft den Vergleich automatisch auf, sobald Referenz-Snapshots existieren.

7.15 Reference-Pipeline via `librasher-cli`

Ursprünglich war geplant, RetroArch auf das Original-Preset `crt-royale.slangp` laufen zu lassen und dort Per-Pass-Snapshots zu greifen. Zwei Probleme machten den Weg unpraktikabel: (1) der Image-Display-Core (nötig, um eigene Test-PNGs als “Spiel” zu laden) wird für macOS nicht mehr ausgeliefert; (2) RetroArch hat keine eingebaute Per-Pass-Snapshot-Funktion – intermediate Framebuffer sind nur über Apitrace-Hacks oder Custom-Patches zugänglich.

Lösung: `librasher-cli` aus dem `librasher`-Projekt (SnowflakePowered/librasher) – die Rust-Reimplementation der Slang-Shader-Pipeline, die mittlerweile auch von RetroArch selbst genutzt wird. Sie läuft auf Metal als nativem Backend, nimmt PNG rein und schreibt PNG raus, und exposed intermediate Stages über den Schalter `--passes-enabled N`.

Aufruf. Für jeden Test-Input und vier Vergleichs-Stages (1, 2, 8, 12 aktivierte Passes – entsprechend unseren eigenen Stages `pass1`, `pass2`, `pass3`, `final`):

```
librasher-cli render \
  --preset      vendor/slang-shaders/crt/crt-royale.slangp \
  --image       tests/inputs/colorbars.png \
  --out         tests/reference/colorbars/02-pass2.png \
  --dimensions  256x768 \
  --passes-enabled 2 \
  --runtime     metal
```

Das Skript `tests/capture_reference.sh` kapselt diesen Aufruf in einer Schleife über alle 8 Test-Inputs $\times$ 4 Stages und schreibt 32 Reference-PNGs nach `tests/reference/<pattern>/`.

Encoding-Konvention. Beim ersten Vergleichslauf zeigte sich ein systematischer Brightness-Offset bei `pass2` ($\Delta E \approx 13\text{--}19$). Genauere Diagnose: `librasher-cli` exportiert die Per-Pass-Snapshots aus sRGB-Framebuffer-Pässen (`srgb_framebuffer<N>=true` im Preset) als *raw linear* PNG-Bytes – PNG-Byte = linear $\times 255$ – und *nicht* als sRGB-encodierte Display-Werte. Nur die finale Pass 11 (ohne `srgb_framebuffer`-Flag) bekommt die echte Display-Encoding.

Unser SwiftRunner exportierte zunächst alle Stages mit Display-Gamma encodiert. Mit einem zusätzlichen Helper-Kernel `pass_linear_export` (nur Saturate, keine Gamma-Anwendung) für die intermediates und Beibehaltung von `pass_final_encode` ausschließlich für `04-final.png` ist die Konvention nun symmetrisch.

ΔE -Ergebnis nach Encoding-Fix. Tabelle 9 zeigt: `pass2` ist **mathematisch äquivalent** zur Slang-Referenz auf allen 8 Test-Inputs ($\overline{\Delta E}_{2000} \leq 0.04$, SSIM = 1.000). Die residuellen Maximalwerte ≈ 1.8 liegen unterhalb der Wahrnehmungsschwelle und stammen aus Single-Pixel-Interpolations-Artefakten am Bildrand. Damit ist die erste Phase-3-Validierung erfolgreich: ein portierter Pass exakt unter dem Akzeptanzkriterium $\overline{\Delta E} < 2.0$ verifiziert.

Pattern	$\overline{\Delta E}_{2000}$	P95	Max	SSIM
colorbars	0.01	0.00	1.81	1.000
gradient	0.04	0.33	1.79	1.000
grid	0.01	0.00	1.79	1.000
horiz_lines	0.00	0.00	1.79	1.000
solid_black	0.00	0.00	0.00	1.000
solid_gray	0.04	0.32	0.32	1.000
solid_white	0.01	0.00	1.79	1.000

Tabelle 9: Pass 2 (Vertical Scanlines) – ΔE_{2000} -Werte gegen das Slang-Original.
Akzeptanzkriterium $\overline{\Delta E} < 2.0$ deutlich erfüllt.

Pass 3 – Mask-LUT-Iteration. Ein zweiter Iterationsschritt am Pass 3 hat dann den Maskenpfad auf LUT-Sampling umgestellt: das Phosphor-Raster wird nicht mehr prozedural generiert, sondern aus `TileableLinearApertureGrille15Wide8And5d5Spacing.png` (512×512 , mipmapped) gesampelt – genau wie im Slang-Original. Drei Detailpunkte mussten dabei korrekt zusammenkommen: (1) die LUT wird als linearer (nicht sRGB-codierter) Texturwert geladen, da die "TileableLinear*.pngKonvention bereits lineare Pixelwerte vorsieht; (2) die Mipmap-Kette muss auf der GPU erzeugt werden, denn ohne sie zeigt sich bei feinen Triadenpitches sichtbares Subpixel-Aliasing; (3) Compute-Shader haben keine impliziten Bildschirm-Derivatives (`ddx/ddy` ist Fragment-only) – die UV-Gradienten müssen daher analytisch via `gradient2d()` an den Sampler übergeben werden, sonst bleibt der Sampler bei Mipmap-Level 0 stehen und das Raster bleibt aliasiert.

Zusätzlich nutzt der Slang-Default `mask_sample_mode_desired = 0` (lese aus `MASK_RESIZE`, dem Output der Slang-Passes 5–6). Slang-Pass 5+6 wurden in Iteration 15 portiert, der Mode-0-Sampling-Pfad ist jedoch nicht bit-exakt vs. librashader (siehe Phase 4 Limitationen). Damit unser Pass 3 stabil mit der Slang-Referenz vergleichbar bleibt, setzen wir beim Reference-Capture `--params mask_sample_mode_desired=1` – das aktiviert in Slang den Hardware-Resample-Branch, den wir in MSL ebenfalls als Default verwenden.

Mit diesen drei Anpassungen liegt Pass 3 jetzt im Bereich $\overline{\Delta E}_{2000} = 0.36 \dots 2.89$ (Tabelle 10, SSIM 0.96...0.99) gegen die Slang-Referenz – für die meisten Patterns unter dem Akzeptanzkriterium von 2.0. Die verbleibende Restdivergenz auf brillanten Mustern (`solid_white`: 2.89, `colorbars`: 1.85) erklärt sich durch die nicht voll portierte horizontale Beam-Filterung (`sample_rgb_scanline_horizontal`, Quilez/Lanczos2) – die Halation- und Bloom-Beiträge sind in den Folge-Iterationen 13–14 ergänzt worden.

Pattern	$\overline{\Delta E_{2000}}$	P95	Max	SSIM
colorbars	1.85	6.18	10.81	0.980
gradient	0.93	4.66	10.39	0.982
grid	0.36	3.46	10.16	0.990
horiz_lines	1.49	6.11	10.52	0.971
solid_black	0.00	0.00	0.00	1.000
solid_gray	0.66	2.08	2.92	0.979
solid_white	2.89	8.89	10.52	0.961

Tabelle 10: Pass 3 (Apply Mask, LUT-Pfad) – ΔE_{2000} -Werte gegen das Slang-Original mit `mask_sample_mode_desired=1`. Ohne LUT (prozedurale Aperture-Grille) liegen die Werte um den Faktor 10–20 höher.

Final. Für die noch nicht vollständig portierte Final-Stage bleibt $\overline{\Delta E_{2000}} = 5 \dots 27$, weil uns Slang-Passes 2–6 (Bloom-Approx + Halation V/H + Mask-Resize), 8–10 (Brightpass + Bloom V/H) und 11 (Geometry/AA) fehlen. Diese Werte werden mit jedem weiteren portierten Pass abnehmen und sind als Fortschrittsmessung in den nächsten Iterationen zentral.

7.16 Dateien im Projekt

Datei	Beschreibung
RetroVisor/GPU/CrtRoyale.metal	Metal Compute Kernels (Pass 1 + Pass 2 + Final)
RetroVisor/Shaders/CrtRoyale.swift	Swift-Integrationsklasse + Snapshot-Export
RetroVisor/Shaders/ShaderLibrary.swift	Registrierung als 3. Shader
crt-royale-msl/src/CrtRoyalePass1Linearize.metal	Standalone-Mirror Pass 1
crt-royale-msl/src/CrtRoyalePass2VerticalScanlines.metal	Standalone-Mirror Pass 2
crt-royale-msl/src/CrtRoyalePass3ApplyMask.metal	Standalone-Mirror Pass 3 (LUT + Fallback)
crt-royale-msl/src/CrtRoyalePass4GeometryAA.metal	Standalone-Mirror Pass 4 (Geometry + Border)
crt-royale-msl/src/CrtRoyaleBloomApprox.metal	Standalone-Mirror BLOOM_APPROX (bilinear)
crt-royale-msl/src/CrtRoyaleBrightpass.metal	Standalone-Mirror BRIGHT-PASS
crt-royale-msl/src/CrtRoyaleBloomVH.metal	Standalone-Mirror Bloom V/H + Reconstitute
crt-royale-msl/tests/SwiftRunner/	Headless Validierungs-Runner (Swift Package)
crt-royale-msl/tests/inputs/generate.py	Generator deterministischer Test-PNG
crt-royale-msl/tests/validate.sh	Master-Skript der Pipeline (Build → Run → Analyze)
crt-royale-msl/tools/analyze.py	Statistischer Pass-Analyzer (Sanity-Checks)
crt-royale-msl/tools/compare.py	Pixel-Diff: ΔE_{2000} + SSIM + Heatmap
crt-royale-msl/tests/capture_reference.sh	Reference-Snapshots via libshader-cli
crt-royale-msl/tests/compare_all.sh	Batch-Diff aller (Pattern, Pass)-Tupel

Tabelle 11: Projektdateien

8 Validierungskonzept

8.1 Methodik

Die Validierung erfolgt durch **quantitativen Per-Pass-Vergleich** zwischen dem Slang-Original und unserer MSL-Portierung. Beide Pipelines bekommen exakt dieselben Test-PNGs als Input; die Original-Pipeline läuft via `libshader-cli` (Metal-Backend), unsere via dem headless `SwiftRunner`. Die Snapshot-Stages `pass1`, `pass2`, `pass3`, `final` werden über `--passes-enabled` an den Stellen abgegriffen, an denen unsere reduzierte Pipeline äquivalente Berechnungen liefert.

1. Identische Eingabebilder für beide Pipelines (`tests/inputs/*.png`, deterministisch generiert)
2. Identische Parameter-Defaults (`crt_gamma`, `lcd_gamma`, `mask_type`, etc.)
3. Beide Pipelines schreiben ihre Per-Pass-Snapshots nach getrennten Verzeichnis-Bäumen
4. Pixel-Differenz pro (Input, Pass) via `tools/compare.py` (ΔE_{2000} + SSIM + Heatmap)
5. `tests/compare_all.sh` aggregiert die Ergebnisse zu einer Tabelle pro Mode
6. Akzeptanzkriterium: mittlerer $\Delta E_{2000} < 2.0$ (kaum sichtbarer Unterschied) für jeden portierten Pass

8.2 Testmuster

Die **240p Test Suite** (NES-ROM) bietet standardisierte Testbilder:

Testmuster	Validiert
Color Bars	Farbgenauigkeit der Gamma-Transformation
Grid Pattern	Scanline-Alignment und Geometrie
Gradient Ramps	Banding-Artefakte bei Gamma-Konvertierung
Checkerboard	Phosphormasken-Alignment
Solid White	Bloom/Halation-Verhalten
Scrolling Content	Interlace-Bobbing und zeitliche Stabilität

Tabelle 12: Testmuster und validierte Aspekte

9 Evaluierung (Phase 4)

9.1 Performance-Messung

Methodik. Der `SwiftRunner-Bench-Modus` (`-bench N`) wickelt die volle CRT-Royale-Pipeline (12 Slang-Passes + Pass 4 Geometry-AA) in eine Closure und führt sie $N = 100$ -mal aus. Jede Iteration erstellt einen frischen `MTLCommandBuffer`, encodiert alle Compute-Kernels, ruft `commit()` + `waitUntilCompleted()` auf und misst `gpuEndTime - gpuStartTime` in Millisekunden. Die ersten 3 Iterationen sind Warmup und werden aus der Statistik ausgeschlossen.

Hardware: Apple M2 (8-Core GPU). Test-Input: Color-Bars-Pattern (256×192 Pixel).

Konfiguration	Auflösung	p50	p95	FPS (p50)
Default (geom_mode=0, mode=1)	256×768	0.95 ms	2.00 ms	1049
8× Y-Upscale	256×1536	1.73 ms	2.02 ms	579
Demo-Modus (Curvature + AA)	256×768	1.18 ms	1.41 ms	848
Demo + 8× Upscale	256×1536	1.13 ms	2.01 ms	886

Tabelle 13: Frame-Time-Statistik für $N = 100$ Iterationen. Alle Konfigurationen bleiben weit unter dem 60-FPS-Budget (16.67 ms).

Beobachtungen.

- Default-Pipeline auf 1.0 ms $\sim$ 1050 FPS. Echtzeit-tauglich mit grosszügigem Headroom.
- Curvature + 16-tap Catmull-Rom-AA kosten nur +25 % (0.95 $\rightarrow$ 1.18 ms). Apple’s GPU parallelisiert die Multi-Sample-Loads effizient.
- Verdopplung der Y-Auflösung steigert die GPU-Zeit nur um $\sim 60\%$ – BLOOM_APPROX (fix 320 × 240), Halation V/H (fix 320 × 240) und Mask-Resize V/H (fix 64 × 48/16 × 48) skalieren sublinear.

9.2 Speicherverbrauch (VRAM)

Die Pipeline hält 11 Zwischen-Texturen (alle `rgba16Float` für HDR-Headroom) plus 6 Mask-LUTs (3 große mipmapped, 3 kleine) gleichzeitig im GPU-Speicher. Bei Viewport 256 × 768:

Kategorie	Anzahl	Größe	Anteil
Zwischen-Texturen (<code>rgba16Float</code> , 8 B/Pixel)	11	9.66 MB	70 %
Mask-LUTs groß (512 ² mipmapped, <code>bgra8Unorm</code>)	3	4.00 MB	29 %
Mask-LUTs klein (64 ² , <code>bgra8Unorm</code>)	3	48 KB	< 1 %
Summe		13.71 MB	

Tabelle 14: VRAM-Footprint bei Viewport-Auflösung 256 × 768. Bei 8×-Y-Upscale (256 × 1536) steigt die Summe auf ~ 23 MB.

Die Bloom-Kette dominiert mit drei Viewport-skalierten `rgba16Float`-Texturen (BRIGHT-PASS, BLOOM_V, BLOOM_FINAL) zu je 1.5 MB. Die fest-dimensionierten Zwischenstufen (BLOOM_APPROX, Halation V/H, Mask-Resize V/H) tragen zusammen unter 2 MB bei. Für ein *Real-Time-Shader-Pipeline* auf modernen mobilen GPUs ist das eine sehr leichte Last; selbst eine M1-Klasse-GPU mit Unified Memory hat hier > 99 % Headroom.

9.3 Qualitätsvergleich vs. Slang-Original

Pixelgenaue Validierung gegen die Slang-Reference via `librashader-cli`. Default-Settings, `mask_sample_mode=`

Pattern	$\overline{\Delta E_{2000}}$ (04-final)	SSIM
colorbars	1.73	0.889
gradient	3.19	0.942
grid	3.30	0.930
horiz_lines	5.04	0.974
solid_black	0.00	1.000
solid_gray	3.00	0.987
solid_white	3.02	0.917

Tabelle 15: Default-Pipeline ΔE_{2000} + SSIM vs. Slang-Reference. Akzeptanzkriterium $\overline{\Delta E} < 2.0$ wird auf `colorbars` und `solid_black` voll erreicht; `horiz_lines` (5.04) ist der höchste verbliebene Wert, getragen von der nicht voll bit-exakten Mask-Resize-Kalibrierung.

9.4 Qualitativer Vergleich vs. Sankara / CRT-Easy

Headless-Compare gegen Sankara und CRT-Easy ist im aktuellen Setup nicht direkt automatisierbar (beide haben keinen SwiftRunner). Algorithmischer / qualitativer Vergleich:

Aspekt	CRT-Royale (dieser Port)	Sankara	CRT-Easy
Pipeline	12-Pass Multi-Stage	MPS-basiert (Resample + Blur + Dilation)	Single-Pass-Kernel
MSL-Zeilen	1560	281	218
Phosphor-Maske	LUT + Lanczos-Resize	Prozedurale Dot-Mask	Prozedurale RGB-Stripes
Beam	Generalized Gaussian ($\beta \in [2, 4]$)	Gauss-Variante	Triangular/Step
Halation/Bloom	2× separable Gauss + Re-constitute	MPS-Gauss + Dilation	—
Curvature	Sphere-Raycaster (full Slang Mode 1)	Optional (MPS)	—
AA	16-tap Catmull-Rom-Cubic	MPS-basiert	—
Performance	1.0–1.2 ms / Frame (M2, 256×768)	nicht gemessen	nicht gemessen

Tabelle 16: Vergleich der Shader-Architektur. CRT-Royale ist die mit Abstand reichhaltigste Implementierung; CRT-Easy ist optimiert auf minimalen Code, Sankara liegt dazwischen.

CRT-Royale liefert die höchste visuelle Treue zur echten CRT-Optik (Subpixel-Rendering, mehrlagiges Bloom, Glas-Reflektion), zum Preis von $\sim 7\times$ mehr Shader-Code als CRT-Easy. Die Performance bleibt auf Apple Silicon trotzdem komfortabel im Echtzeit-Bereich.

9.5 Bekannte Limitationen

1. **Mask-Resize V/H Mode-0-Pfad:** nicht bit-exakt vs. `librasher-cli`. Grund: `derived-settings-and-Defines` (vermutlich `GEOMETRY_EARLY`, `ANISOTROPIC_TILING_COMPAT_*`) werden in `librasher` anders aufgelöst als unsere statische Hand-Trace – die `mask_resize_tile_size`-Berechnung produziert in `librasher` deutlich kleinere Tiles. Bypass: `mask_sample_mode=1` (hardware-resample) ist der Default und ΔE -validiert.
2. **Curvature-Branch nicht ΔE -validiert:** benötigt einen eigenen Reference-Snapshot-Satz mit aktivierter Curvature. Sphere-Raycaster ist mathematisch port-äquivalent zu Slang Mode 1.
3. **AA-Filter-Konfigurabilität:** Catmull-Rom-Cubic fixiert (Slang exponiert 10 Filter $\times$ 11 Sample-Counts).
4. **Geometrie-Modi 2 (Sphere_Alt) + 3 (Cylinder):** nicht portiert. Sphere ist die typische Wahl.

10 Projektplanung

10.1 Phasenübersicht

Phase	Bezeichnung	Inhalt	Status
1	Portierung	Übersetzung aller 12 Passes GLSL $\rightarrow$ MSL	Abgeschlossen
2	Integration	Einbindung in RetroVisor, UI-Settings, Texturen	Abgeschlossen
3	Validierung	Per-Pass-Vergleich mit <code>librasher-cli</code> -Reference	Abgeschlossen (11 von 12 Passen)
4	Evaluierung	SSIM, ΔE , Frame-Time, Vergleich vs. Sankara/CRT-Easy	Abgeschlossen

Tabelle 17: Phasenübersicht

10.2 Detailplan Phase 1: Portierung

Pass	Beschreibung	Status
0	Linearize CRT Gamma + Bob Fields	✓ Abgeschlossen
1	Vertical Scanlines (Beam-Distribution)	✓ Abgeschlossen
2	Bloom Approx (4 $\times$ 4 Gauss, 320 $\times$ 240)	✓ Abgeschlossen
3	Halation Vertical (9-tap Gauss)	✓ Abgeschlossen
4	Halation Horizontal $\rightarrow$ HALATION_BLUR	✓ Abgeschlossen
5	Mask Resize Vertical (Lanczos)	✓ Abgeschlossen (Mode-0 nicht bit-exakt)
6	Mask Resize Horizontal $\rightarrow$ MASK_RESIZE	✓ Abgeschlossen (Mode-0 nicht bit-exakt)
7	Apply Mask + Halation (MASKED_SCANLINES)	✓ Abgeschlossen
8	Brightpass (Area-Bloom-Extract)	✓ Abgeschlossen
9	Bloom Vertical	✓ Abgeschlossen
10	Bloom Horizontal + Reconstitute	✓ Abgeschlossen
11	Geometry + AA + Final Encode	✓ Abgeschlossen

Tabelle 18: Pass-Portierungsstatus

11 Muss- und Kann-Kriterien

Muss-Kriterien:

- M1** Vollständige Portierung aller 12 Passes nach MSL.
- M2** Lauffähige Integration in RetroVisor als wählbarer Shader.
- M3** Echtzeit-Fähigkeit: ≥ 30 FPS bei 1080p-Eingabe.
- M4** Visuell korrekte Darstellung der CRT-Kerneffekte (Scanlines, Maske, Gamma).
- M5** Dokumentierte Vergleichsbilder (GLSL-Original vs. MSL-Port).
- M6** Quellcode unter Open-Source-Lizenz auf GitHub verfügbar.

Kann-Kriterien:

- K1** ≥ 60 FPS bei 1080p (optimiert für Apple Silicon).
- K2** Alle ~ 45 Parameter über RetroVisor-UI konfigurierbar.
- K3** SSIM > 0.95 gegenüber dem GLSL-Original.
- K4** Unterstützung aller drei Phosphormasken-Typen.
- K5** half-Präzisions-Variante für reduzierte GPU-Last.
- K6** Performance-Vergleich mit CRT-Easy als Baseline.

12 Glossar

Begriff	Definition
CRT	Cathode Ray Tube – Röhrenbildschirm
MSL	Metal Shading Language – Apples GPU-Programmiersprache
GLSL	OpenGL Shading Language – plattformübergreifende GPU-Sprache
Slang	Shader-Format der libretro-Sammlung (GLSL-basiert)
Compute Shader	GPU-Programm ohne Bindung an die Grafikpipeline
Kernel	Metal-Bezeichnung für eine Compute-Shader-Funktion
Threadgroup	Gruppe von GPU-Threads, die gemeinsam ausgeführt werden
Scanline	Horizontale Bildzeile eines CRT-Monitors
Phosphormaske	RGB-Subpixel-Anordnung auf der CRT-Bildschirmfläche
Bloom	Lichtüberstrahlung (Glow) um helle Bildbereiche
Halation	Lichtstreuung durch die Glasscheibe des CRT-Monitors
Interlacing	Halbbildverfahren (gerade/ungerade Zeilen alternierend)
Bob Deinterlacing	Deinterlacing durch abwechselnde Halbbild-Anzeige
Gamma	Nichtlineare Helligkeitskurve (Helligkeit = Signal^γ)
Konvergenz	Ausrichtung der drei Elektronenstrahlen (R, G, B)
Lanczos	Hochwertiges Resampling-Verfahren (Sinc-basiert)
SSIM	Structural Similarity Index – Qualitätsmetrik für Bilder
ΔE	Wahrnehmungsbasierte Farbdifferenz (CIE-Farbraum)
RetroVisor	macOS-App für GPU-basierte Echtzeit-Bildeffekte
libretro	API-Standard für Emulator-Frontends (RetroArch)

Tabelle 19: Glossar